

## C++/Modern C++ (some background info useful later on)

- a) typedef vs using
- b) explicit, from C++20: explicit(bool-expression)
- c) template class vs typename
- d) #define vs constexpr, std::string\_view
- e) python vs C++
- f) move semantics
- g) std::variant (vs std::visit alongside lambda overloads) and std::optional - C++17

a) typedef vs using  
typedef existing\_type new\_name; // readability + portability  
since C++11: using new\_name = existing\_type; // supports template aliases

Example #1 - Basic Template Alias -> an alias for a std::map where the key is always a std::string, but the value type remains flexible

```
#include <iostream>
#include <map>
#include <string>

// Template Alias: Fixes the key as std::string, leaves Value open
template <typename T>
using StringMap = std::map<std::string, T>;

int main() {
    // StringMap<int> expands to std::map<std::string, int>
    StringMap<int> playerScores;
    playerScores["Alice"] = 1500;
    playerScores["Bob"] = 1200;

    // StringMap<double> expands to std::map<std::string, double>
    StringMap<double> currencyRates;
```

```

currencyRates["EUR"] = 1.08;

std::cout << "Alice's score: " << playerScores["Alice"] << "\n";
}
/*
// The old, complicated C++98 way using typedef
template <typename T>
struct StringMapWrapper {
    typedef std::map<std::string, T> type;
};
// Usage is NOT as clean:
StringMapWrapper<int>::type playerScores;

```

// **cannot** just write `typedef std::map<std::string, T> type;` outside of a `struct` because `typedef` does **not** accept template parameters directly

```

*/

```

Example #2 - **Fixing Certain** Parameter/s

```

#include <vector>
#include <array>

```

// 1. Alias that locks the size of a `std::array` to 3 (e.g., **3D Coordinates**)

```

template <typename T>
using Point3D = std::array<T, 3>;

```

// 2. Alias that locks the **type** but leaves the size open

```

template <size_t N>
using IntArray = std::array<int, N>;

```

```

int main() {
    Point3D<float> floatPoint = {1.0f, 2.0f, 3.0f}; // std::array<float, 3>
    IntArray<5> fiveNumbers = {1, 2, 3, 4, 5};      // std::array<int, 5>
}

```

```
}
```

b) `explicit`

```
{ from C++20 (doesn't necessarily mean it's always useful): explicit(bool-expression) }
```

In a C++ class, the `explicit` keyword is a specifier used to prevent the compiler from performing **unintended implicit type conversions**.

By default, any **constructor** that can be called **with a single argument** is considered a "converting constructor". This means the compiler can automatically use it to turn a value of another type into an object of your class—often leading to **subtle bugs**.

What it specifically does

- **Blocks Implicit Conversions:** It forces you to manually (explicitly) cast or call the constructor when you want to create an object from a different type
- **Disallows Copy-Initialization:** You can no longer use the `=` syntax for initialization if it requires a type conversion
- **Requires Direct-Initialization:** You must use parentheses or braces to create the object

```
class Buffer {
public:
    // Without 'explicit', this is a converting constructor
    Buffer(size_t capacity) { /* allocates memory */ }
};

void SendData(const Buffer& b) { /* ... */ }

int main() {
    // This looks like we are sending the character 'A'
    // But it actually creates a Buffer of size 65 (ASCII for 'A') and sends that!
    SendData('A');

    // This also works but is confusing: it looks like an assignment
    Buffer b = 100;
```

```

}
.. vs ..

class Buffer {
public:
    explicit Buffer(size_t capacity) { /* ... */ }
};

int main() {
    // SendData('A'); <-- COMPILER ERROR: cannot implicitly convert char to Buffer
    // Buffer b = 100; <-- COMPILER ERROR: prevents this "assignment-style" creation
    // You MUST be explicit now:
    Buffer b(100);           // Direct initialization works
    SendData(Buffer(100));   // Explicitly creating a temporary object works
}

```

`explicit(bool-expression)` - a specifier introduced in C++20 that allows you to make a constructor or conversion operator "conditionally explicit"

- Behavior: If the boolean expression inside the parentheses evaluates to `true` at compile-time, the function is marked `explicit`. If it evaluates to `false`, the function is non-explicit (implicit).
- Default: Using the keyword `explicit` alone (without a boolean expression) is equivalent to writing `explicit(true)`.
- Requirement: The expression must be a constant expression of type `bool`.

.. particularly useful in template programming, where you want a constructor to be explicit only if the underlying types `require` it.

```

template<typename T>
struct Wrapper {
    // Only explicit if the type T cannot be implicitly converted from an int
    explicit(!std::is_convertible_v<int, T>) Wrapper(int val) {}
};

Wrapper<double> w1 = 10;    // OK: implicit (explicit(false))

```

```
Wrapper<std::string> w2{1}; // OK: direct-init
// Wrapper<std::string> w3 = 1; // Error: explicit(true) triggered
```

c) `template <class T>` vs `template <typename T>` .. this is a longer discussion ..

d) `#define` vs `constexpr`, `std::string_view`

Hardware properties are inherently static - shifting configuration checks from simulation runtime to compile-time

`#define` vs `constexpr` .. example related to Bus Configuration:

```
#define BUS_WIDTH 32      vs      static constexpr size_t BusWidth = 32;
```

**Zero-Allocation Naming:** Passing strings into `sc_module` names historically caused `std::string` allocations. Can use `std::string_view` for high-performance, immutable string handling.

e) python vs C++

- 1) Python's `OrderedDict` and C++'s `std::map` (often called an ordered map) are built on entirely different data structures with completely different performance characteristics; `std::unordered_map`
- 2) Memory Management & Object Mutability
  - The Trap: Passing mutable objects to functions can cause unintended side effects.
  - C++: Everything is passed by value (copied) unless you explicitly use references (&) or pointers (\*).
  - Python: Everything is passed by object reference. Modifying a `list` or `dict` inside a function mutates the original object outside the function.
  - **Bonus Pitfall:** Default arguments in Python are evaluated once when the function is defined, not when it is called. Writing `def append_to(element, target=[])` will reuse the exact same list across multiple function calls.
- 3) Number Overflow vs. Arbitrary Precision
  - The Trap: Forgetting about overflow in C++ or underestimating performance differences.
  - C++: Standard integers (`int`, `long long`) have strict size limits (e.g., 32-bit or 64-bit). You must frequently use the modulo operator (`% 10^9 + 7`) to prevent integer overflow.
  - Python: Integers have arbitrary precision (automatically grow to fit the number). While you will never overflow, extremely large number math slows down drastically because it switches from hardware-level math to software-level math.

#### 4) Direct Indexing & Iterator Invalidation

- The Trap: Modifying a collection while looping through it.
- C++: Erasing an element from a `std::vector` or `std::map` while iterating over it invalidates the iterator, causing undefined behavior or crashes.
- Python: Modifying a dictionary's keys while iterating over it explicitly throws a `RuntimeError: dictionary changed size during iteration`. Modifying a list while looping won't crash, but it will silently skip elements because indices shift.

#### 5) Division Behavior

- The Trap: Getting the wrong data type or rounding the wrong direction on negative numbers.
- C++: The `/` operator on two integers performs truncation toward zero (e.g., `5 / 2 = 2` and `-5 / 2 = -2`).
- Python: The `/` operator always returns a float (`5 / 2 = 2.5`). The `//` operator performs floor division, which rounds toward negative infinity (e.g., `5 // 2 = 2` but `-5 // 2 = -3`).

#### 6) Built-in Data Structure Namesakes

- The Trap: Assuming structures with similar names share the same performance profiles.

| Concept            | C++ Structure                    | Python Structure               | The Critical Difference                                                                                                    |
|--------------------|----------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Dynamic Array      | <code>std::vector</code>         | <code>list</code>              | Python's <code>list</code> holds references => worse cache locality than a dense C++ vector.                               |
| Hash Set           | <code>std::unordered_set</code>  | <code>set</code>               | Both $O(1)$ , but Python's <code>set</code> has significantly higher memory overhead per element.                          |
| Max-Heap           | <code>std::priority_queue</code> | <code>heapq</code>             | C++ defaults to a Max-Heap. Python's <code>heapq</code> defaults to a Min-Heap.                                            |
| Double-Ended Queue | <code>std::deque</code>          | <code>collections.deque</code> | C++ deque uses blocked memory chunks (good indexing). Python deque uses a doubly-linked list of blocks ( $O(n)$ indexing). |

While both `std::deque` in C++ and `collections.deque` in Python provide  $O(1)$  time complexity for adding or removing items from either end (`push_back/append` and `push_front/popleft`), they differ drastically when it comes to accessing items by index.

#### f) Move Semantics (`std::move` and Rvalue References)

TLM modeling prioritizes speed, simulating millions of transactions per second. Deep copying data structures across sockets completely ruins simulation performance.

- Zero-Copy Transactions: Move semantics are used to pass heavy data blocks (like custom networking packets, bus bursts, or large payloads) between a Master and Slave interface without duplicating data memory.
- Registers & Buffers: Moving data directly into internal module registers via `std::move` bypasses costly allocations.

#### g) `std::variant` (`std::visit` alongside `lambda overloads` even cleaner?) and `std::optional` (C++17)

`std::variant` Example: Multi-Protocol Heterogeneous Bus

A common problem in TLM 2.0 modeling is designing a shared SystemC interconnect or router that must pass different transaction packet formats (e.g., an AXI4 configuration write, a tilelink coherent cache message, or a custom debug trace).

`std::variant` creates a type-safe union. The packet takes up only as much memory as its largest possible type, and the compiler guarantees you can never read the wrong format.

```
#include <systemc>
#include <variant>
#include <vector>
#include <iostream>

// Protocol Type A: Standard Memory Command
struct MemoryCommand {
    uint64_t address;
    size_t burst_length;
};

// Protocol Type B: Cache Coherency Message
struct CoherencyMsg {
```

```

    uint32_t tile_id;
    bool invalidate_line;
};

// Protocol Type C: Debug/Trace Payload
struct DebugTrace {
    std::string message;
};

// Define the heterogeneous type alias
using InterconnectPayload = std::variant<MemoryCommand, CoherencyMsg, DebugTrace>;

class BusRouter : public sc_core::sc_module {
public:
    SC_CTOR(BusRouter) {}

    // Process an incoming transaction packet from the network on chip
    void route_packet(const InterconnectPayload& packet) {
        std::cout << "@" << sc_core::sc_time_stamp() << " [Router] Packet Received: ";

        // std::holds_alternative checks what type is currently inside the variant
        if (std::holds_alternative<MemoryCommand>(packet)) {
            // std::get extracts the specific type securely
            auto mem_cmd = std::get<MemoryCommand>(packet);
            std::cout << "Type: Memory Command | Target Addr: 0x"
                      << std::hex << mem_cmd.address << std::dec << "\n";
        }
        else if (std::holds_alternative<CoherencyMsg>(packet)) {
            auto cache_msg = std::get<CoherencyMsg>(packet);
            std::cout << "Type: Cache Coherency | Origin Tile: " << cache_msg.tile_id
                      << " | Invalidate? " << (cache_msg.invalidate_line ? "Yes" : "No") << "\n";
        }
    }
};

```



```

    else if (std::holds_alternative<DebugTrace>(packet)) {
        auto trace = std::get<DebugTrace>(packet);
        std::cout << "Type: Debug String | Log: \"" << trace.message << "\"\n";
    }
};

```

`std::optional` Example: Hardware Interrupt & Valid Lines

In hardware, optional lines like an Interrupt Request (IRQ) line or a Valid/Ready handshake either carry a value or remain completely inactive.

Instead of using a hacky "magic number" (like `-1` or `0xFFFF`) to mean "no interrupt," `std::optional` explicitly models whether a signal wire is driven or floating.

```

#include <systemc>
#include <optional>
#include <iostream>

// A system interrupt payload
struct InterruptEvent {
    uint32_t source_id;
    uint32_t priority;
};

class PeripheralModule : public sc_core::sc_module {
public:
    // The interrupt line might hold data, or it might be completely empty (floating)
    std::optional<InterruptEvent> irq_line = std::nullopt;

    SC_CTOR(PeripheralModule) {
        SC_THREAD(process_logic);
    }
}

```

```

private:
    void process_logic() {
        // Scenario A: No interrupt active
        check_interrupt_status();

        // Scenario B: Hardware fires an interrupt
        sc_core::wait(10, sc_core::SC_NS);
        irq_line = InterruptEvent{ .source_id = 5, .priority = 2 };
        check_interrupt_status();
    }

    void check_interrupt_status() {
        // .has_value() safely checks if the wire is being driven
        if (irq_line.has_value()) {
            // Access fields cleanly using arrow -> or dereference *
            std::cout << "@" << sc_core::sc_time_stamp()
                << " [CPU] Servicing IRQ from Source: " << irq_line->source_id
                << " (Priority: " << irq_line->priority << ")\n";

            // Clear the line (hardware clear/ack)
            irq_line = std::nullopt;
        } else {
            std::cout << "@" << sc_core::sc_time_stamp()
                << " [CPU] No active interrupts on the line.\n";
        }
    }
};

```